

Reviewer Pack — Additive Energy of Fourier Coefficients Bounds SOS Refutatio...

Entry 16b99cc1e016 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 04:33:03 UTC

Additive Energy of Fourier Coefficients Bounds SOS Refutation Size

Entry ID: 16b99cc1e016 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 04:33:03 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): SOS Refutation Size for CSPs

Statement:

For a random 3-CNF formula Φ with n variables and m clauses, the additive energy E of its Fourier coefficients satisfies $E = \Theta(m^2 / k)$, where k is the SOS refutation size. If $E < m^2 / (2k)$, then Φ has a SOS refutation of size $\leq k$.

Rationale (proposer’s reasoning):

Additive energy captures the structure of Fourier coefficients, which encode the correlation between clauses. Lower energy implies more structured interactions, enabling faster SOS refutations. This bridges additive combinatorics with CSP proof complexity via Fourier analysis.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 31ee894db2cb986a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_3cnf(n, m):
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = [random.choice(variables) * (2 * random.randint(0, 1) - 1)
                  for _ in range(3)]
        clauses.append(clause)
    return clauses

def walsh_hadamard_transform(f, n):
    N = 1 << n
    for s in range(n):
        mask = 1 << (n - s - 1)
        for x in range(N):
            if x & mask:
                f[x] -= f[x ^ mask]
            else:
                f[x] += f[x ^ mask]
        for x in range(N):
            f[x] /= math.sqrt(2)

def additive_energy(f, n):
    E = 0
    N = 1 << n
    for i in range(N):
        for j in range(i + 1, N):
            E += abs(f[i] * f[j])
    return E

def sos_refutation_size(clauses):
    # Simple heuristic: tree depth of DPLL-based search
    max_depth = 0
    stack = [(clauses, [])]
    while stack:
        remaining_clauses, assignment = stack.pop()
```

```

        if not remaining_clauses:
            max_depth = max(max_depth, len(assignment))
            continue
        clause = random.choice(remaining_clauses)
        for var in clause:
            new_assignment = assignment + [var]
            new_remaining = [c for c in remaining_clauses if not any(v in c for v in new_assignment)]
            stack.append((new_remaining, new_assignment))
    return max_depth

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 20
    m = 5 * n
    clauses = generate_3cnf(n, m)
    f = [0] * (1 << n)
    for clause in clauses:
        for x in range(1 << n):
            if all((x >> abs(v) - 1) & 1 == v // abs(v) for v in clause):
                f[x] += 1
    walsh_hadamard_transform(f, n)
    E = additive_energy(f, n)
    k = sos_refutation_size(clauses)
    if k == 0:
        return {
            "metric_name": "E / (m2 / k)",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "SOS refutation size is zero"
        }
    metric_value = E * k / m**2
    conjecture_holds = abs(metric_value - 1) < 0.1
    counterexample = "" if conjecture_holds else f"E = {E}, m2/k = {m**2/k}"
    return {
        "metric_name": "E / (m2 / k)",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"E < m^2 / (2k)\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing validation of support fraction or counterexample detection. | next: Increase timeout duration and re-run experiment with identical parameters

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	81546
2	preregistration	ollama_remote	qwen3:8b	0	0	20052
3	novelty	ollama_remote	qwen3:8b	0	0	16578
4	novelty	ollama_remote	qwen3:8b	0	0	11151
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13059
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11343
7	judge	ollama_remote	qwen3:8b	0	0	13338

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 167068 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/16b99cc1e016.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/16b99cc1e016.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/16b99cc1e016.tar.gz` (if generated)